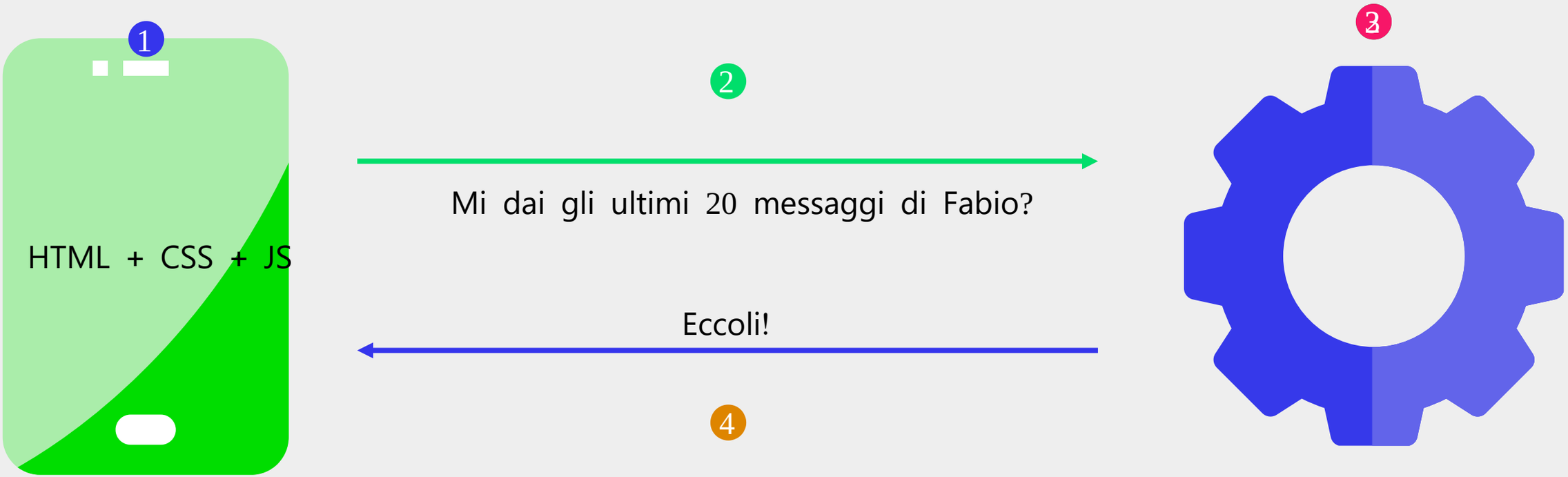
API

Come far dialogare la nostra pagina con l’esterno

Un visione d’insieme

Gestione documenti didattici e Stanza Virtuale



Un visione d’insieme

1. Quali tecnologie aggiuntive servono ?
2. Che forma ha il messaggio da inviare ?
3. Come faccio a chiamare proprio il server che voglio ?
4. Come funziona la ricezione di dati “asincrona”?

Quali tecnologie aggiuntive servono?

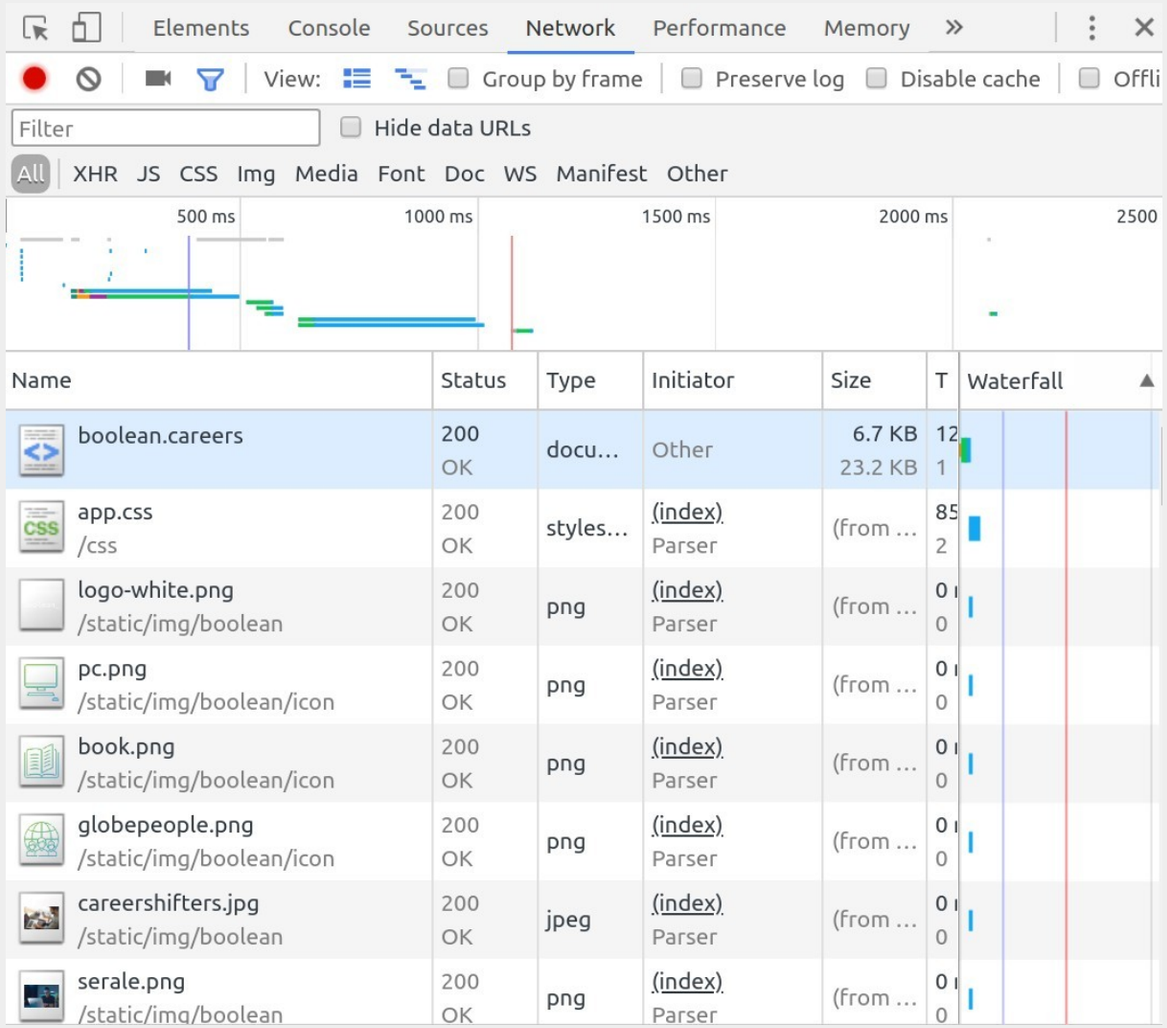
The good news: nessun nuovo linguaggio di programmazione

JavaScript ha la possibilità di fare chiamate a server esterni e gestirne la risposta .
Come spesso accade , possiamo utilizzare una libreria esterna per rendere più semplice il processo.

Per quanto riguarda la possibilità di far comunicare più entità attraverso la rete , dobbiamo utilizzare il protocollo HTTP .
L'HTTP è un protocollo utilizzato come principale sistema per la trasmissione di informazioni sul web,con architetture client server .

La prima implementazione del protocollo HTTP risale al 1990, dalle mani del solito Tim Berners-Lee.
I browser utilizzano HTTP per “mostrare” i siti.

HTTP nel browser



HTTP

Messaggio di richiesta

Il messaggio di richiesta è composto da quattro parti :

HTTP Request

1. Riga di richiesta
2. Header
3. Riga vuota
4. Body (corpo) della richiesta

HTTP - Riga di Richiesta

Messaggio di richiesta

METODO:
identifica il tipo di operazione richiesta dal client.

I due metodi più utilizzati sono GET e POST

(ma esistono anche HEAD P,UT , PATCH,DELETE , OPTIONS, TRACE, CONNECT)

URI:
unified resource identifier è il nome del server che voglio contattare

(www.boolean.careers)

VERSIONE PROTOCOLLO:
1.1 dal 1997 fino al 2014

2.0 dal 2015

HTTP - Header

Messaggio di richiesta

Header
è uno spazio di lunghezza variabile che può contenere diverse informazioni nel formato

Chiave: Valore

User-Agent: una stringa identificativa del browser utilizzato

Accept: una lista di tipi di file che il client si aspetta per la richiesta che ha fatto

Content -Length: la lunghezza della richiesta , in modo che il server possa verificare di averla ricevuta correttamente

Authorization: una stringa che il client invia per identificarsi

Ce ne sono tanti identificati dallo standard HTTP , ma nessuno ci vieta di aggiungerne altri

HTTP - Body

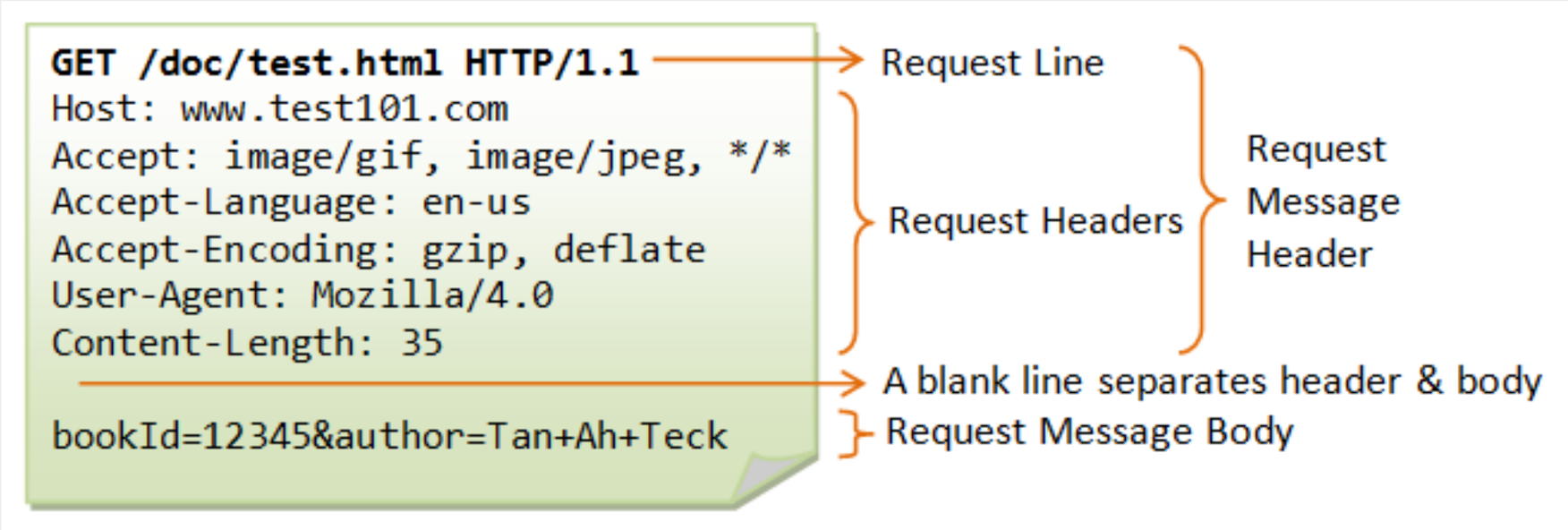
Messaggio di richiesta

Body
il corpo (body) della richiesta non è altro che la parte che identifica la risorsa da richiedere .

www.boolean.careers/corso/serale

URI **HTTP BODY**

HTTP Request



Header di una HTTP Request

▼ **Request Headers** view source

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9,it;q=0.8
Cache-Control: max-age=0
Connection: keep-alive
Cookie: olfsk=olfsk7101296722461601; hblid=K5DXZK51o3FUQyhO9G7zM0O92JE2B2RB; _ga=GA1.2.1617061698.1510940911; cookiebar=CookieAllowed; _gid=GA1.2.1466887918.1523788595; _hjIncludedIn
Sample=1; XSRF-TOKEN=eyJpdiI6ImlKbitVWGZhUnk0QStjMGJ3NDBnV2c9PSIsInZhbHVlIjoiYVZtQVlwT1I2OTdIdmFJczhUOUhuQ0E1M0RyM1JxVTBudzBaWVpwRXpUNytkbjVCZXUzMGQ4cXBrSG9UMWVxZ2s4TzRJZkVkTURjZ2Ux
bWRYTUh1Y3c9PSIsIm1hYyI6IjZkZGJiM2U0NTc2MTI5OTc4ZGUzODI3ZGFiMzE1ZDY0ZjRlZGEwZjhjMWRhMTBkZGVlNzNkYjQyMjk1NzZkYTQifQ%3D%3D; laravel_session=eyJpdiI6IlNMclBBRXlOMjFCb2w2elFxRUNIZEE9PSI
sInZhbHVlIjoib2tWUFpaK3drYWxyQ2FOTzJ3SVQrMmZFUFZvd290U1hjUTJ0S2NsVjNramZaU0RkZExKc3lvSkRRR2E5RFB4SkV3YXltSFA0eUlxQWp6XC92NEF3TlZnPT0iLCJtYWMiOiI5ZmRjZmVkMDlkM2VkMjU3MmY2NTVmZTc3ODc0
MDk0MzkzMmY0OTg3Yjg2YWY0MmQwNjAwYmU3MGViNmRhNDFiIn0%3D; _drip_client_4803864=vid%253Dcda084201b0a013605d20e2e8fb74b86%2526pageViews%253D15%2526sessionPageCount%253D2%2526lastVisited
At%253D1523788605321%2526weeklySessionCount%253D9%2526lastSessionAt%253D1523788594894%2526form%255B118563%255D%255Bauto_open%255D%253D1523788625%2526form%255B118563%255D%255Bmanual_
close%255D%253D1523788631%2526form%255B118563%255D%255Bmanual_open%255D%253D1523635098
DNT: 1
Host: boolean.careers
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/65.0.3325.181 Safari/537.36

HTTP Response

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Encoding: UTF-8
Content-Length: 138
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close


<html>
<head>
  <title>An Example Page</title>
</head>
<body>
  Hello World, this is a very simple HTML document.
</body>
</html>
```

Response Header

Response Body

HTTP Response codes

La risposta contiene un codice che indica lo stato della risposta , i più noti e utili stati sono:

- **200 - OK**. Il server ha fornito correttamente il contenuto nella sezione body .
- **301 - Moved Permanently**. La risorsa che abbiamo richiesto non è raggiungibile perché è stata spostata in modo permanente .
- **400 - Bad Request**. La risorsa richiesta non è comprensibile al server.
- **404 - Not Found**. La risorsa richiesta non è stata trovata .
- **500 - Internal Server Error**. Il server non è in grado di rispondere alla richiesta per un suo problema interno .

HTTP Response codes

Più in generale :

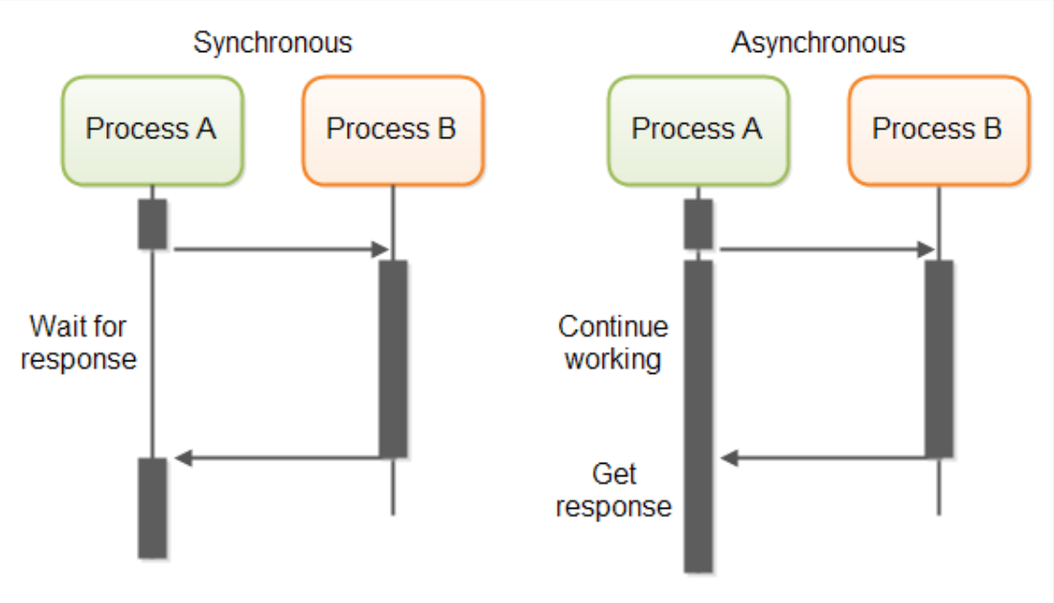
- **1xx - Informational** (messaggi informativi)
- **2xx - Successful** (la richiesta è stata soddisfatta)
- **3xx - Redirection** (non c'è risposta immediata , ma la richiesta è sensata e viene detto come ottenere la risposta)
- **4xx - Client error** (la richiesta non può essere soddisfatta perché sbagliata)
- **5xx - Server error** (la richiesta non può essere soddisfatta per un problema interno del server)

Come facciamo una chiamata HTTP?

JS + HTTP = Ajax!

Anche i programmatori web possono fare chiamate HTTP :
utilizzando JavaScript !

La tecnologia **AJAX**,
Asynchronous JavaScript And XML,
permette di fare chiamate asincrone .



Con Ajax quindi posso inviare/ricevere dati a/da un server senza ricaricare pagina .

Quale altro modo abbiamo visto, ma mai utilizzato veramente, per inviare dati da una pagina HTML?

JS + HTTP = Ajax!

XMLHttpRequest

Questa è una (parziale) implementazione di una chiamata Ajax in JavaScript vanilla.

```
var xhttp = new XMLHttpRequest();

xhttp.onreadystatechange = function () {
 if (this.readyState == 4 && this.status == 200) {
   // Action to be performed when the document is ready;
 }
};

xhttp.open("GET", "www.boolean.careers/api/random/boolean", true);
xhttp.send();

```

Vue?

Axios

Utilizzeremo la libreria **Axios** che ci permetterà di eseguire chiamate asincrone .
(da inserire nell'head: https://cdnjs.cloudflare.com/ajax/libs/axios/0.20.0/axios.min.js)

```
axios.get('https://flynn.boolean.careers/exercises/api/random/boolean')
 .then(function (response) {
   const result = response.data;
 });
```

Server e client devono parlare la stessa lingua :

deve esserci quindi uno standard per comunicare dati.

Che lingua sarà?

JSON e XML

Cosa si inviano client e server?

Esistono due standard :

XML (eXtensible Markup Language) è una linguaggio di markup (simile all'HTML) che stabilisce un meta linguaggio per passare dati strutturati.

JSON (JavaScript Object Notation) utilizza la notazione che conosciamo per definire gli oggetti in JavaScript

XML

eXtensible Markup Language

```xml
<?xml version="1.0" encoding="UTF-8"?>
<utenti>
 <utente anni="20">
   <nome>Ema</nome>
   <cognome>Princi</cognome>
   <indirizzo>Torino</indirizzo>
 </utente>
 <utente anni="54">
   <nome>Max</nome>
   <cognome>Rossi</cognome>
   <indirizzo>Roma</indirizzo>
 </utente>
</utenti>
```

JSON

JavaScript Object Notation

```
{
 "users": [
   {
   "name": "Ema",
   "cognome": "Princi",
   "indirizzo": "Torino",
   "anni": 20
   },
   {
     "name": "Max",
     "cognome": "Rossi",
     "indirizzo": "Roma",
     "anni": 54
   }
 ]
}
```

XML vs JSON

JSON e XML sono due strumenti diversi per risolvere lo stesso problema

```xml
<?xml version="1.0" encoding="UTF-8"?>
<utenti>
 <utente anni="20">
   <nome>Ema</nome>
   <cognome>Princi</cognome>
   <indirizzo>Torino</indirizzo>
 </utente>
 <utente anni="54">
   <nome>Max</nome>
   <cognome>Rossi</cognome>
   <indirizzo>Roma</indirizzo>
 </utente>
</utenti>
```

```json
{
 "users": [
   {
   "name": "Ema",
   "cognome": "Princi",
   "indirizzo": "Torino",
   "anni": 20
   },
   {
     "name": "Max",
     "cognome": "Rossi",
     "indirizzo": "Roma",
     "anni": 54
   }
 ]
}
```

JS + JSON =❤️

Scrivendo in JavaScript, JSON per noi è la scelta più ovvia per due motivi:

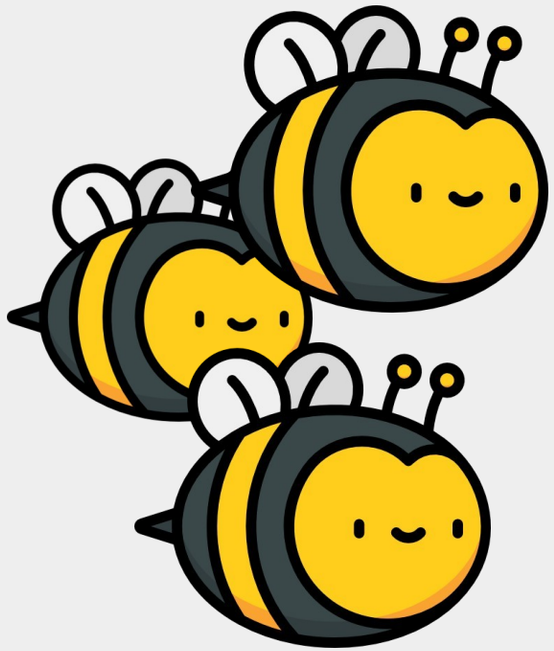
1. C'è corrispondenza nella notazione, se sai scrivere un oggetto sai scrivere e comprendere JSON
2. Una volta arrivato la risposta dal server in formato JSON è molto semplice trasformare la risposta (che è semplicemente testo) in un oggetto JavaScript, con JSON parse().

```
var obj = JSON.parse('{ "name":"John", "age":30, "city":"New York"}');
```

API

Application Programming Interface

Con API si intende un insieme di procedure rese disponibili ad un programmatore per svolgere un insieme coerente di operazioni.
Non è una terminologia legata alla programmazione web .



- Esempi di API che non hanno a che fare con il mondo internet :
- Le Windows API di Microsoft
- Le API Carbon e Cocoa per MacOS
- Mantle per DirectX e OpenGL
- …

È un termine generico che indica delle operazioni che il programmatore può fare , rese disponibili nascondendo parte della complessità sottostante .

Query string

Passare al server dati

Un modo per passare dati al server è attraverso l'URI aggiungendo al termine una **querystring**

querystring

www.dominio.it ?email=fabio@boolean.careers&title=CEO

Query string

Passare al server dati

www.dominio.it ?email=luca.lambia@gmail.com&title=MR

1 ? indica che inizia la query string

2 chiave=valore

3 ogni chiave-valore è separata da un &

Postman App

Testiamo le chiamate ad una Api

https://www.getpostman.com/downloads/